Homework 1:Theoretical

1. **From logits to softmax, and then to NLL loss**

In a language model, the model first outputs a **predicted embedding**, which is then multiplied by the **vocabulary embedding matrix** to obtain a score (logit) for each candidate word. Then, **softmax** is applied to obtain a probability distribution. Finally, the **negative log-likelihood (NLL)** is computed for the ground-truth next word.

Assume the current context is:

> **"the dog chased the"**

The vocabulary contains only 4 candidate words:

$$V = [\text{cat},\ \text{ball},\ \text{garden},\ \text{banana}]$$

The predicted embedding produced by the model is:

$$h = [1,\ 2]$$

The vocabulary embedding matrix (where each **column** corresponds to one word in the vocabulary above) is:

$$E = \begin{bmatrix} 2 & 1 & 0 & -1 \\ 1 & 0 & 2 & 1 \end{bmatrix}$$

The logits are computed by:

$$z = h^\top E$$

That is, the predicted embedding is dotted with each word embedding to obtain the corresponding logit.

Assume the true next word is **cat**.

- (1) Calculate the logits of the four candidate words. Write out each element of vector z, and state which word has the largest logit.
- (2) Calculate the softmax probabilities. You may use the following approximations:

  $$e^4 \approx 54.60, \quad e^1 \approx 2.72, \quad e^0 = 1$$

  Compute:

  $$P(\text{cat}),\ P(\text{ball}),\ P(\text{garden}),\ P(\text{banana})$$

  and verify that the probabilities sum to approximately 1.
- (3) Calculate the NLL loss for this sample. Since the true next word is **cat**, compute:

  $$\text{NLL} = -\log P(\text{cat})$$

  Round your result to **three decimal places**.
- (4) Suppose the true next word is changed to **garden**. Without recomputing the entire softmax, answer the following based on your result above:
  - Will the NLL become larger, smaller, or stay the same?
  - What does this indicate?

2. **Suppose we have a trained language model, and the current input context is "The cat".**

The model needs to predict the next **2 tokens**.

The simplified vocabulary includes 5 candidate tokens:

[sat, ate, on, fish, mat]

Given the input context **"The cat".**

Given: The probability distribution output by the model at each step is shown in the table below.

Previous Token	Candidate	Probability
step1: **cat**	**sat**	0.5
	ate	0.4
	on	0.1
step2: **sat**	**on**	0.6
	mat	0.4
step2: **ate**	**fish**	0.9
	on	0.1

Please complete the following three tasks:

- (1) Greedy Search
  - Please write the token chosen at each step.
  - Calculate the **joint probability** of the final generated sequence (i.e., the product of probabilities at each step).
  - What is the final sentence?

- (2) Beam Search (k=2)
  - Assume Beam Width **k=2** (keep the top 2 paths with the highest probabilities).
  - Please list the 2 paths retained after Step 1 and their cumulative probabilities.
  - Please list the highest-scoring path and its cumulative probability after Step 2.
  - What is the final sentence?

- (3) Temperature Sampling
  - Consider only the Step 1 probability distribution: **[0.5, 0.4, 0.1]**.
  - Please compute the new probabilities of **sat** and **ate** when Temperature T=0.5.

    (Hint: $P_{new} \propto P_{old}^{1/T}$, renormalization is required.)
  - Briefly explain: When T **decreases**, does the probability distribution become **sharper** or **flatter**? What effect does this have on the generation results?

3. **Recurrent neural networks, also known as RNNs, are a type of neural network used for sequence modelling. In this question, we will think conceptually about how an RNN processes information.**

Consider the simple RNN architecture shown in Figure1. The inputs $x_1, x_2, \ldots X_T$ are vectors in $\mathbb{R}^{d_{in}}$,the hidden states $h_1, h_2, \ldots h_T$ are vectors in $\mathbb{R}^{d_{hidden}}$, and the outputs $y_1, y_2, \ldots, y_T$ are vectors in $\mathbb{R}^{d_{out}}$.The hidden states and outputs are given by recurrence relations:

$$\mathbf{h}_t = \phi_h \left(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h \right) \quad (1)$$

$$\mathbf{y}_t = \phi_y \left(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y \right) \quad (2)$$

The functions $\phi_h(\cdot)$ and $\phi_y(\cdot)$ are arbitrary element-wise non-linearities. The RNN has three weight matrices $\mathbf{W}_h$, $\mathbf{W}_x$ and $\mathbf{W}_y$ and two bias vectors $\mathbf{b}_h$ and $\mathbf{b}_y$.

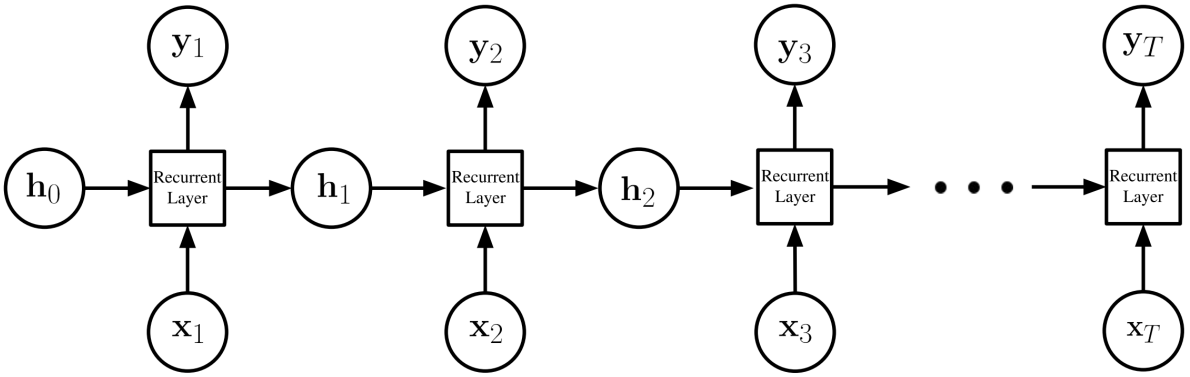

Figure 1. A simple RNN architecture. At time t, an RNN computes a hidden state $\mathbf{h}_t$ based on the current input $\mathbf{x}_t$ and prior hidden state $\mathbf{h}_{t-1}$. The RNN also spits out an output $\mathbf{y}_t$.

For simplicity, in this question we will set the initial hidden state $\mathbf{h}_0$= 0, we will set the non-linearity ϕ_h to the identity $\phi_h(h)$ = h and we will set the biases $\mathbf{b}_h$ = 0 and $\mathbf{b}_y$ = 0. Under this simplification, after one time step T=1: $\mathbf{h}_1 = \mathbf{W}_x \mathbf{x}_1$ and $\mathbf{y}_1 = \phi_y(\mathbf{W}_y \mathbf{W}_x \mathbf{x}_1)$.

- (1) Derive formulae for hidden state $\mathbf{h}_2$ and output $\mathbf{y}_2$ in terms of the weight matrices $\mathbf{W}_y, \mathbf{W}_x, \mathbf{W}_h$and inputs $\mathbf{x}_1, \mathbf{x}_2$.

- (2) Derive formulae for hidden state $\mathbf{h}_3$ and output $\mathbf{y}_3$ in terms of the weight matrices $\mathbf{W}_y, \mathbf{W}_x, \mathbf{W}_h$and inputs $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$.

- (3) Derive formulae for hidden state $\mathbf{h}_T$ and output $\mathbf{y}_T$ in terms of the weight matrices $\mathbf{W}_y, \mathbf{W}_x, \mathbf{W}_h$and inputs $\mathbf{x}_1, \ldots, \mathbf{x}_T$.

- (4) Suppose the sequence length T is very long. What do you notice about the contribution of the first input $\mathbf{x}_1$ to the last output $\mathbf{y}_T$ of the RNN?

- (5) Introducing Attention Mechanism

  To address the issue identified in question (4), suppose we introduce an **Attention Mechanism** at the output layer. Instead of relying solely on the final hidden state *h***T* , the model computes a context vector *c* as a weighted sum of all hidden states:$c = \sum_{i=1}^{T} \alpha_i h_i$* where α_i are attention weights ($\sum \alpha_i = 1$). The final output is now $y_{attn} = \phi_y(W_y c)$.

  - **(a)** Consider the influence of the first input x_1 on the new output y_{attn} . Through which specific hidden state does x_1 have the most **direct** influence on y_{attn} ? (Hint: Compare the path from x_1 to y_{attn} vs x_1 to y_T).

- **(b)** Based on your derivation in **(4)**, explain why this Attention architecture helps mitigate the long-term dependency problem when T is very large.

4. **Under what conditions is Negative Log-Likelihood equivalent to the Cross-Entropy loss function? Provide the derivation process.**

Homework (Lab) 1: Foundations of Deep Learning

Covering Labs 1-3: Python/NumPy, PyTorch/Autograd, Preprocessing/Output

Submission Instructions

Academic Honesty: This assignment must be completed individually. You may discuss high-level concepts with classmates, but sharing code or solutions is not permitted.

Statement on AI tools: Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about language models, but using it directly to solve the problem is prohibited. We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub CoPilot) in your IDE when completing assignments (though non-AI autocomplete is fine).

1. **Environment Setup:** Use the provided `pyproject.toml` (or your own environment).
2. **Implementation:** Complete the implementation in `p1_numpy_ops.py`, `p2_grad_reversal.py`, and `p3_softmax_topk.py`.
3. **Constraints:**
   - Do not modify function signatures or the provided code structure.
   - Only implement code inside the designated `TODO` blocks.
4. **Submission:**
   - Zip **only** your Python implementation files: `p1_numpy_ops.py`, `p2_grad_reversal.py`, and `p3_softmax_topk.py`.
   - Do **not** rename these files.
   - Name your zip file using your student ID (e.g., `1241xxxx.zip`).

Problem 1: NumPy Vectorized Operations (30 pts)

In this problem, you will implement several core numerical operations using NumPy. These operations form the foundation of many deep learning algorithms.

Math vs. Implementation: The summation notation and formulas below provide the formal **mathematical definition** of the operations. You should **not** translate these into loops. Instead, refer to the NumPy Documentation to find the appropriate high-level vectorized functions (e.g., `np.dot`, `np.sum`, broadcasting) that implement these concepts efficiently.

1. **Part A: Basic Operations** (10 pts)
   - `matrix_multiply(A, B)`: Standard matrix multiplication. For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times k}$, the result $C = AB \in \mathbb{R}^{m \times k}$ is defined as:

$$C_{i,j} = \sum_{l=1}^{n} A_{i,l} B_{l,j}$$

   - `normalize_rows(M)`: Normalize each row to have unit L_2 norm. For a row vector v:

$$v_{\text{norm}} = \begin{cases} \frac{v}{\|v\|_2} & \text{if } \|v\|_2 > 0 \\ v & \text{otherwise} \end{cases}$$

- `create_one_hot(indices, num_classes)`: Convert a 1D array of class indices $y \in \{0, ..., K-1\}^N$ into a 2D binary matrix $Y \in \{0,1\}^{N \times K}$ where:

$$Y_{i,j} = \begin{cases} 1 \text{ if } j = y_i \\ 0 \text{ otherwise} \end{cases}$$

2. **Part B: Statistical Operations** (10 pts)
   - `column_standardize(X)`: Standardize features (columns) to zero mean and unit variance. For each column j:

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}$$

     If $\sigma_j = 0$ (constant feature), the column should remain unchanged ($z_{i,j} = x_{i,j}$).
   - `pairwise_l2_distance(A, B)`: Compute the Euclidean distance between every pair of rows from A and B. For a_i from A and b_j from B, the distance $D_{i,j}$ is:

$$D_{i,j} = \sqrt{\|a_i - b_j\|_2^2} = \sqrt{\|a_i\|_2^2 + \|b_j\|_2^2 - 2a_i^T b_j}$$

     Ensure numerical stability by preventing negative values before the square root.

3. **Part C: Softmax & Loss** (10 pts)
   - `softmax_batch(logits)`: Implement numerically stable softmax for a batch of inputs $X \in \mathbb{R}^{N \times K}$. For each row x:

$$\text{softmax}(x)_i = \frac{\exp(x_i - \max(x))}{\sum_j \exp\big(x_j - \max(x)\big)}$$

   - `cross_entropy_loss(probs, targets)`: Compute the mean cross-entropy loss over N samples:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log\big(p_{\{i,y_i\}} + \varepsilon\big)$$

     where $p_{\{i,y_i\}}$ is the predicted probability for the true class y_i. Use a small $\varepsilon = 10^{-12}$ for stability.

Problem 2: Gradient Reversal Layer (40 pts)

Background: Domain Adversarial Neural Networks (DANN)

In many real-world scenarios, the data we use for training (**Source Domain**, e.g., synthetic images) has a different distribution than the data we use during testing (**Target Domain**, e.g., real-world photos). A model trained only on the source domain might fail on the target domain due to this **domain shift**. The goal of **domain adaptation** is to learn features that are robust to these distribution changes.

The DANN Framework

Domain Adversarial Neural Networks (DANN) aim to learn **domain-invariant features**—features that are useful for the main task but do not contain information about which domain the data came from. The architecture consists of:

1. **Feature Extractor (G_f)**: Maps input x to a feature vector f.
2. **Label Classifier (G_y)**: Predicts the class y from f (main task).
3. **Domain Classifier (G_d)**: Predicts the domain d (Source=0 or Target=1) from f (adversarial task).

The key insight is that by training the feature extractor to **confuse** the domain classifier, we force it to learn features that are invariant to the domain shift.

The Role of the Gradient Reversal Layer (GRL)

The GRL is placed between the Feature Extractor and the Domain Classifier. It enables adversarial training through a clever gradient manipulation:

- **Forward Pass:** Identity function: $y = x$.
- **Backward Pass:** Reverses and scales the gradient: $\nabla_x = -\lambda \cdot \nabla_y$.

This creates an **adversarial** relationship:

- The **domain classifier** tries to minimize the domain classification loss.
- The **feature extractor** (due to the reversed gradient) is pushed to maximize this loss.

Why Custom Autograd? Standard automatic differentiation only tracks gradients of forward operations. Since the GRL is an identity function in the forward pass, standard autograd would pass gradients unchanged. The GRL requires custom backward logic that negates and scales gradients—a pattern not achievable with standard layers. This demonstrates why understanding and implementing custom autograd functions is essential for advanced deep learning architectures.

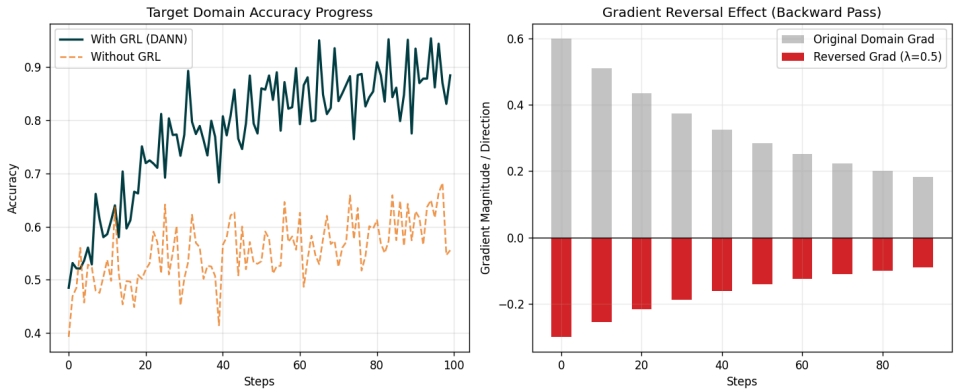

Figure 1: DANN training: GRL enables learning domain-invariant features, improving accuracy on the target domain (left), while reversing and scaling gradients (right).

Implementation Task: DANN Architecture

Your task is to implement the core GRL logic and a complete DANN model architecture in `p2_grad_reversal.py`:

- **Part A (10 pts):** `GradientReversalFunction.forward` — Save `lambda_` to `ctx` and return input unchanged.
- **Part B (15 pts):** `GradientReversalFunction.backward` — Return negated and scaled gradient:

$$-\lambda \cdot \text{grad\_output}$$

  .
- **Part C (5 pts):** `GradientReversalLayer.forward` — Apply the custom function using `GradientReversalFunction.apply`.
- **Part D (10 pts):** `DANNModel` — Implement a two-headed architecture:
  - Shared feature extractor: `nn.Linear(784, 256)` -> `ReLU`
  - Label head: `nn.Linear(256, 10)` (predicts digit class 0-9)
  - Domain head: `GRL(lambda_)` -> `nn.Linear(256, 2)` (adversarial domain prediction)

Problem 3: Softmax, Temperature & Top-K Sampling (30 pts)

Background: The LLM Decoding Pipeline

When a Large Language Model (LLM) generates text, it does not simply pick the single most likely word at every step (Greedy Search). Instead, we **sample** from the probability distribution produced by the model. This stochasticity allows for more diverse and creative text generation.

Key Components

1. **Temperature Scaling (T):** Before applying softmax, we divide the raw **logits** by a temperature parameter T:
   - **Low Temperature ($T < 1$):** Sharpens the distribution, making high-probability tokens more likely (more predictable/deterministic).
   - **High Temperature ($T > 1$):** Flattens the distribution, making all tokens more equally likely (more creative/random).
   - **Temperature $T = 1$:** Standard softmax, no scaling.
2. **Numerically Stable Softmax:** The standard softmax formula $\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$ can cause numerical overflow for large inputs. We use the stable variant:

$$\text{softmax}(x)_i = \frac{\exp(x_i - \max(x))}{\sum_j \exp(x_j - \max(x))}$$

3. **Top-K Filtering:** To avoid sampling very unlikely tokens, we keep only the top k most probable tokens and set all others to zero probability before renormalizing.

Implementation Task

Implement the following logic in `p3_softmax_topk.py`:

- `manual_softmax(logits, temperature)`: Numerically stable softmax with temperature scaling.
- `top_k_filtering(logits, k)`: Keep only top-k logits, mask others with -infinity, then apply softmax.
- `sample_with_temperature(logits, temperature, k, num_samples)`: Full sampling pipeline combining temperature scaling, top-k filtering, and multinomial sampling.

References

- CS231n NumPy Tutorial (NumPy Basics & Vectorization)
- D2L: Softmax Regression (zh) (Mathematical Foundations of Softmax)
- Ganin & Lempitsky (2015): Unsupervised Domain Adaptation by Backpropagation (DANN & GRL Background)

— End of Assignment 1 —